

ASSIGNMENT 4 – RAG AGENT FOR CYBERSECURITY RESULTS

First we deploy the pods ai agent as a pod and the emulated network topology as a pod, we confirm this by running `kubectl get pods -A` (the pods in namespace rag-assistant) are the deployed pods.

First we build the images by running the build image shell script:

```
allen@allen:~/cybersecurity_automation_agent/scripts$ ./build-image.sh
=> [internal] load build definition from Mininet.Dockerfile
=> => transferring dockerfile: 620B
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [internal] load .dockerignore
=> => transferring context: 138B
=> [1/5] FROM docker.io/library/ubuntu:22.04@sha256:4f838adc7181d9039ac795a7d0aba05a9bd9ecd480d294483169c5def98
=> => resolve docker.io/library/ubuntu:22.04@sha256:4f838adc7181d9039ac795a7d0aba05a9bd9ecd480d294483169c5def98
=> [internal] load build context
=> => transferring context: 1.83kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] RUN apt-get update      && apt-get install -y --no-install-recommends      python3      py
=> [4/5] COPY . /app
=> [5/5] RUN chmod +x /app/network_sim/mininet_network_api.py
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:a775b3c2814909e2c593a907cbc5b61ee02d2a3d2367abd222a2bf3945b7024d
=> => exporting config sha256:860e04f611f1d84ba7fbb33e4e260a8d5574619026783c4ff127c764cf5fb3a2
=> => exporting attestation manifest sha256:3e59e1d8c12092fef9c223804a5240e6b1a06267b892bd0bb827133d7600ca15
=> => exporting manifest list sha256:29b2cc6149308afc4aa6e2e98d646f3cbfa06879897f0ccbf5cc5b4179e9cfef
=> => naming to docker.io/library/mininet-sim:dev
=> => unpacking to docker.io/library/mininet-sim:dev
Built image: rag-assistant:dev
Built image: mininet-sim:dev
allen@allen:~/cybersecurity_automation_agent/scripts$
```

Now we run the deploy pods shell script to run the ai agent and the mininet emulated network topology:

```

allen@allen:~/cybersecurity_automation_agent/scripts$ ./deploy-k8s.sh
[sudo] password for allen:
unpacking docker.io/library/rag-assistant:dev (sha256:2a185dcc357b74b428fd81d95e789c3f2cf90f8bf3bae615d11be5df2549707d)...done
unpacking docker.io/library/mininet-sim:dev (sha256:29b2cc6149308afc4aa6e2e98d646f3cbfa06879897f0ccbf5cc5b4179e9cfef)...done
namespace/rag-agent created
secret/rag-assistant-secret created
deployment.apps/rag-assistant created
ingress.networking.k8s.io/rag-assistant created
deployment.apps/mininet-sim created
service/mininet-sim created
service/rag-assistant created
deployment.apps/rag-assistant restarted
deployment.apps/mininet-sim restarted
Waiting for deployment "rag-assistant" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "rag-assistant" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "rag-assistant" rollout to finish: 1 old replicas are pending termination...
deployment "rag-assistant" successfully rolled out
deployment "mininet-sim" successfully rolled out
Forwarding from 127.0.0.1:8501 -> 8501
Forwarding from [::1]:8501 -> 8501

Deployment ready in namespace 'rag-agent'.
Port-forward running in background (PID 15617).
Open: http://localhost:8501
Stop tunnel: kill 15617
allen@allen:~/cybersecurity_automation_agent/scripts$ Handling connection for 8501
Handling connection for 8501
Handling connection for 8501

```

We can verify the pods are running in the system:

```

allen@allen:~$ kubectl get pods -A

```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
container-registry	registry-6c9fcc695f-vnn8t	1/1	Running	23 (5m50s ago)	13d
ingress	nginx-ingress-microk8s-controller-k88mb	1/1	Running	23 (5m51s ago)	13d
kube-system	calico-kube-controllers-77bd7c5b-87144	1/1	Running	25 (5m51s ago)	13d
kube-system	calico-node-7sjbr	1/1	Running	25 (5m50s ago)	13d
kube-system	coredns-864597b5fd-pt6d8	1/1	Running	25 (5m50s ago)	13d
kube-system	hostpath-provisioner-756cd956bc-pkhrm	1/1	Running	27 (5m50s ago)	13d
rag-agent	mininet-sim-6bd685f7b5-ns6mg	1/1	Running	0	66s
rag-agent	rag-assistant-7748dcddc5-fkwbc	1/1	Running	0	67s

```

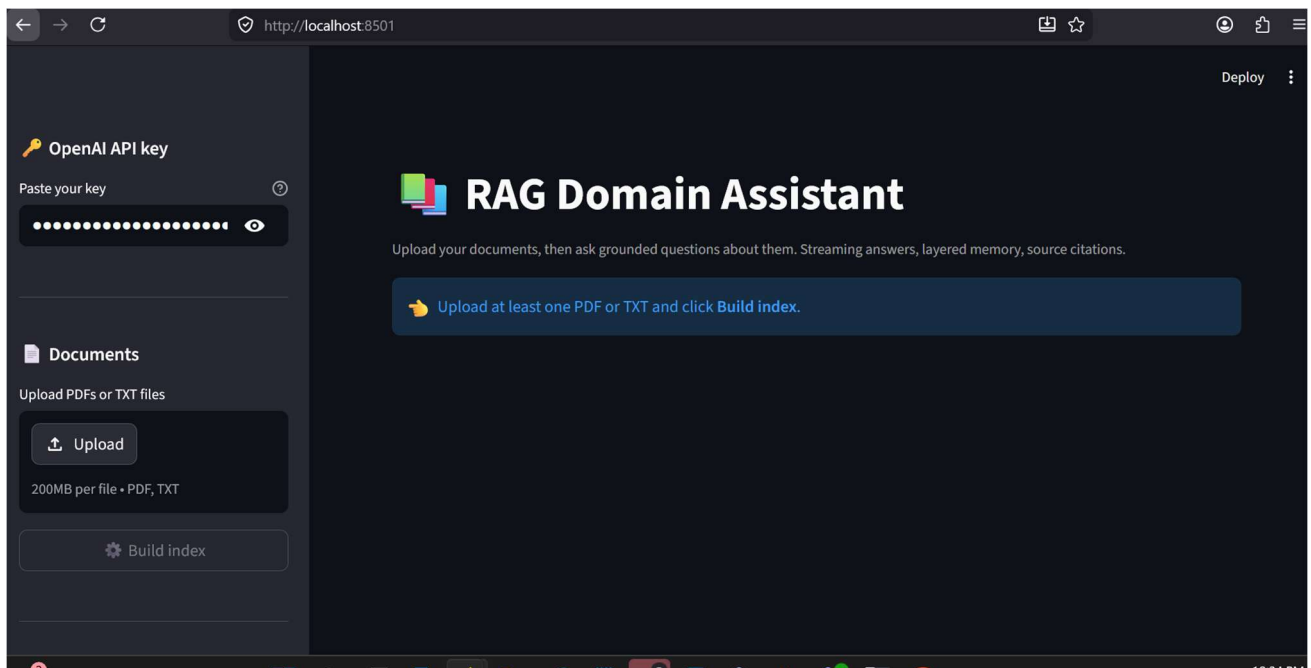
allen@allen:~$

```

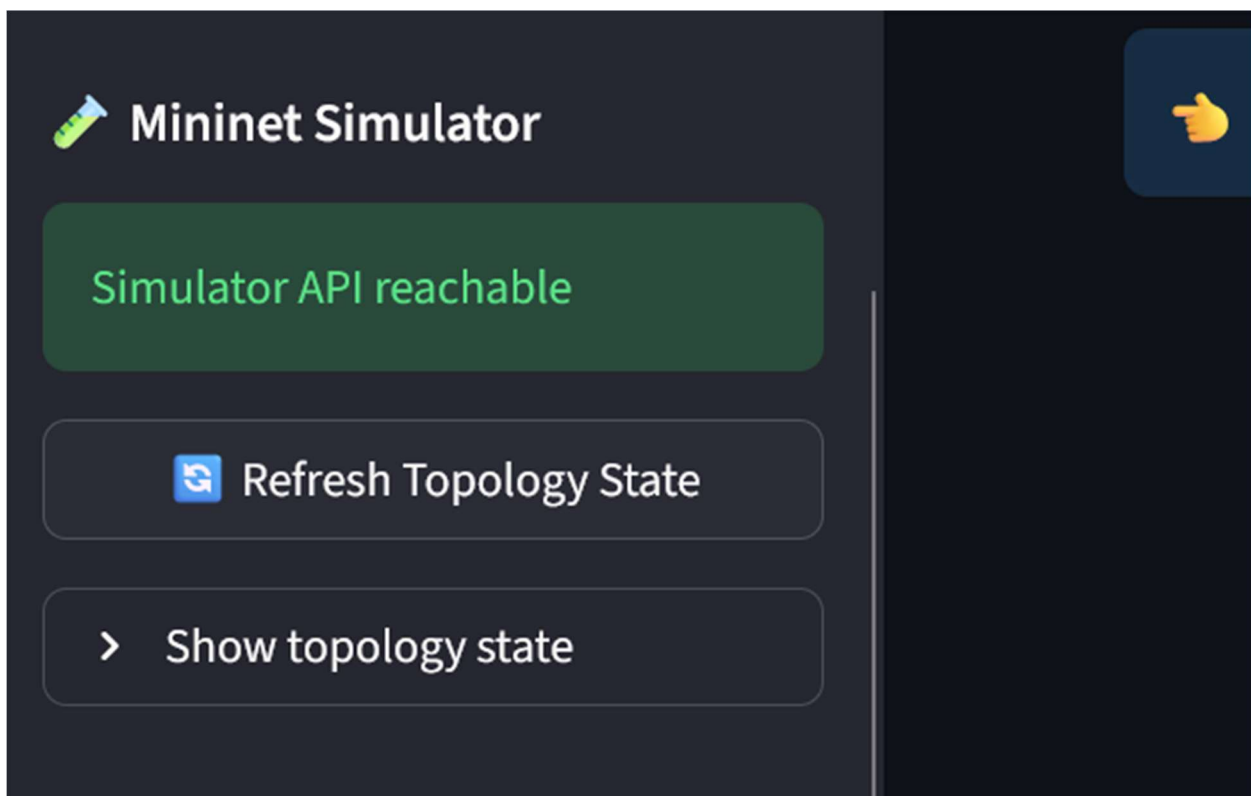
The pods in the namespace 'rag-assistant' are the ones. The rag-assistant-7748... pod is the ai agent and the mininet-sim-6bd6... is the mininet network topology.

The results obtained are shown below:

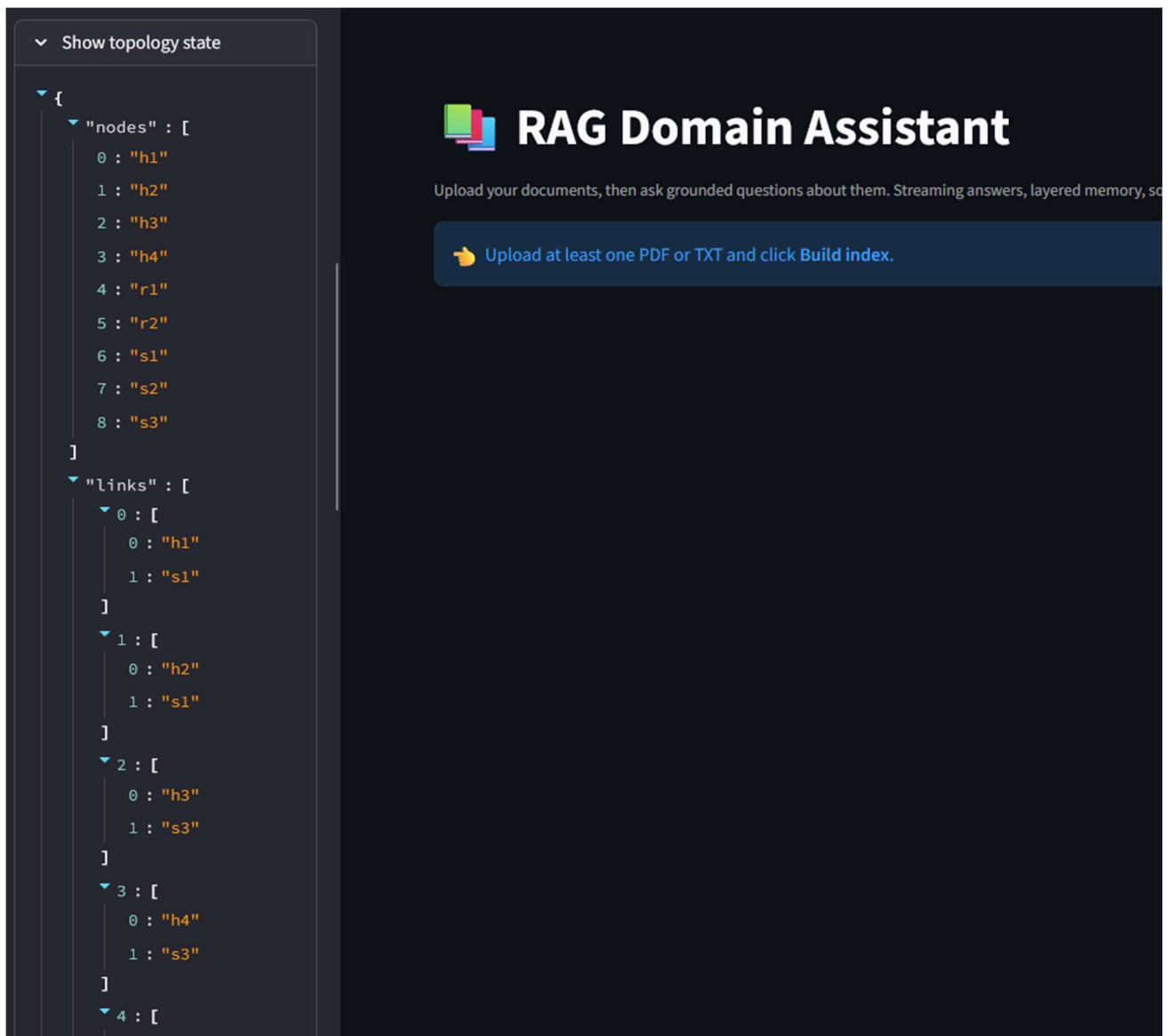
The GUI using streamlit of the RAG AI agent can be seen:



It also displays connectivity to network controller API

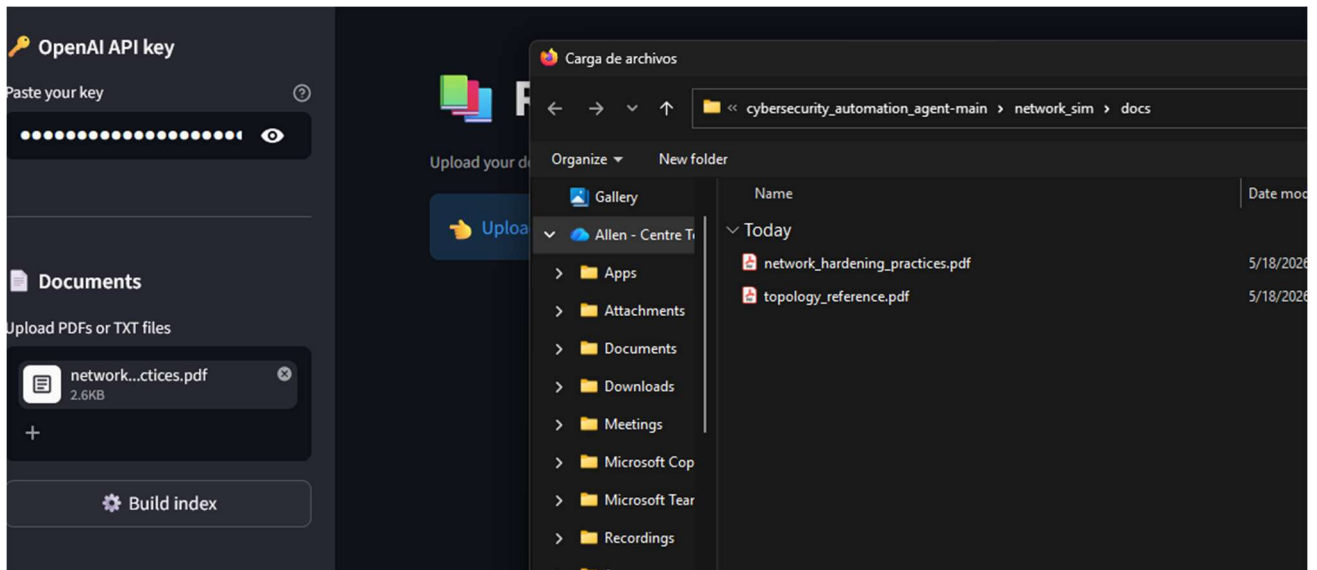


And shows network topology state information

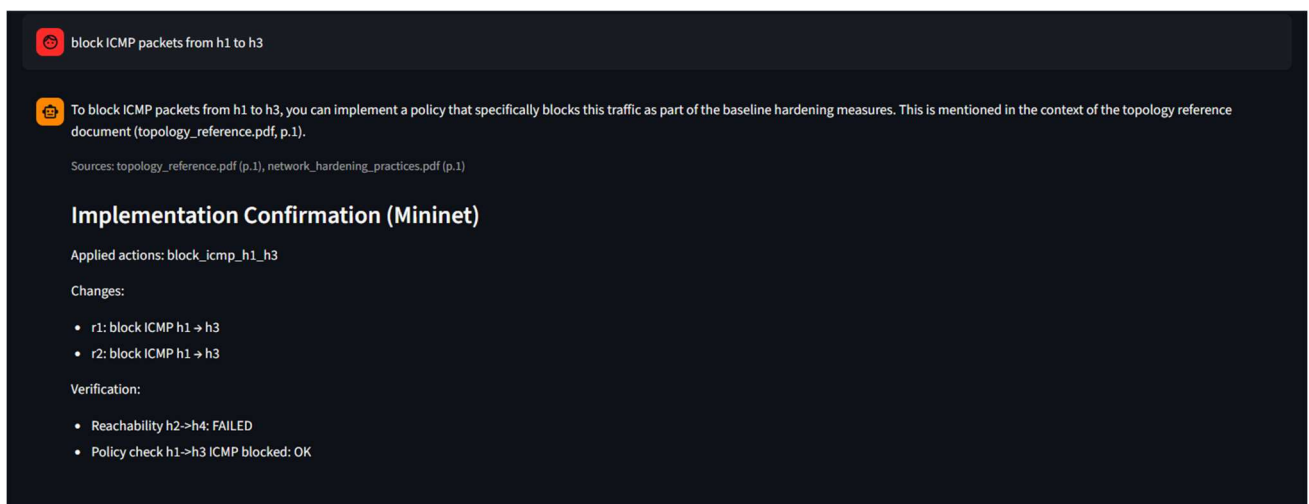


The next step is to upload the documents for cybersecurity operations in the FAISS VectorDB of the ai agent.

We uplod the openai api key and upload the documents into the faiss vectordb of the ai agent, the documents are chunked and converted into embedding vectors and stored in the faiss vector database.



Now we state intent to the ai-agent and the AI agent executes it into the network devices of the connected network topology.



The ai-agent has successfully blocked ICMP traffic. ICMP protocol blocked means the pinging will not be possible anymore. This is confirmed by trying to reach to h4 from h2 and failing.

The ai agent also shows source of reference that is topology_reference.pdf through which it gains contextual understanding on how to block ICMP traffic in the network topology attached to it.